

CRYPTO TRADING BOT

System Design Report

BTC/USDT · ETH/USDT · Binance Paper Trading

PREPARED FOR	Executive Leadership / CEO Review
CLASSIFICATION	CONFIDENTIAL — Internal Use Only
PLATFORM	Binance Testnet & Demo Trading (Paper Orders Only)
INSTRUMENTS	BTC/USDT · ETH/USDT (Spot)
DEVELOPMENT TEAM	Cheng · Gilbert · Grace · ShiYi · sookoon
DATE	June 14, 2026

Table of Contents

1. Executive Summary
2. System Architecture
3. Data Layer
4. Event Engine
5. Regime Detector
6. Strategy Layer
7. Risk Manager
8. Order Management System
9. Portfolio
10. Live Dashboard
11. Backtester
12. Key Design Decisions
13. Current Status and Roadmap

Crypto Trading Bot — System Design Report

Project: Algorithmic Paper-Trading Bot (BTC/USDT · ETH/USDT)

Platform: Binance Testnet / Demo Trading (paper orders only)

Stack: Python 3.12 · asyncio · ccxt · Streamlit · Plotly

Team: Cheng, Gilbert, Grace, ShiYi, sookoon

Table of Contents

1. [Executive Summary](#1-executive-summary) 2. [System Architecture](#2-system-architecture) 3. [Data Layer](#3-data-layer) 4. [Event Engine](#4-event-engine) 5. [Regime Detector](#5-regime-detector) 6. [Strategy Layer](#6-strategy-layer) 7. [Risk Manager](#7-risk-manager) 8. [Order Management System](#8-order-management-system) 9. [Portfolio](#9-portfolio) 10. [Live Dashboard](#10-live-dashboard) 11. [Backtester](#11-backtester) 12. [Design Decisions](#12-key-design-decisions) 13. [Current Status and Roadmap](#13-current-status-and-roadmap)

1. Executive Summary

This report documents the architecture and component logic of an event-driven algorithmic trading bot built as a five-person team project. The system is designed to operate exclusively on paper-trading venues — it never touches a real-money account. Market data may be read from Binance mainnet (public, read-only) while all order execution is routed to a separate paper venue (Binance Testnet or Demo Trading), providing realistic signals without any financial risk.

The bot processes live order-book and OHLCV data, classifies the current market regime, routes signals to the appropriate trading strategy, applies risk controls, and executes paper orders through a lifecycle-managed OMS. A Streamlit dashboard provides real-time monitoring and runtime controls, and a standalone backtester replays the identical strategy and risk stack against historical data to validate performance.

2. System Architecture

2.1 High-Level Overview

The bot is built around a single `asyncio` event queue. No module calls another directly — each registers a handler function that consumes one event and returns a list of new events. The engine routes those back into the queue. This decouples every component and makes it straightforward to add, remove, or test any module in isolation.



2.2 Event Chain

Why this order matters: Portfolio must mark positions before Risk checks drawdown. Regime must update before Strategy generates a signal. Publisher must be last so the snapshot reflects the full tick's state changes.

3. Data Layer

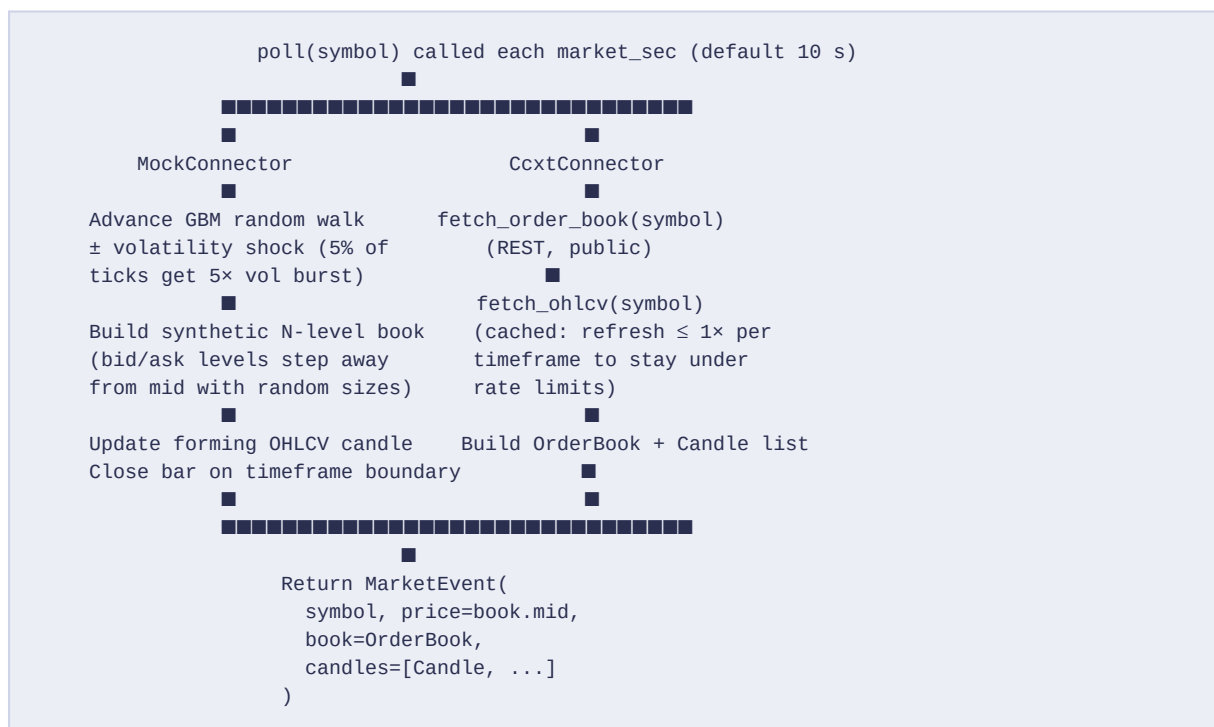
File: `src/part1_data/connector.py`

Owner: Cheng, Gilbert

3.1 Purpose

The data layer is the only entry point for market information. It produces `MarketEvent` objects that carry two data sources per tick: a multi-level order book (microstructure signal and execution reference) and a rolling window of OHLCV klines (the price series for technical indicators).

3.2 Logic Flowchart



3.3 MockConnector — Indicator Warm-Up

A critical design feature: the `MockConnector` pre-generates `kline_limit` (default 100) historical candles at construction time using a geometric Brownian motion walk. This means every strategy's indicator (RSI, Bollinger, MACD, MA) has a full history from tick zero — there is no cold-start period where indicators return NaN in mock mode.

3.4 Rationale

Decision	Reason
Two separate ccxt clients (data + broker)	The data feed can read real mainnet prices (realistic signals) while orders are guaranteed to stay on the paper venue. Merging them would require credentials on the data path and risk accidental production trading.
kline cache refreshed $\leq$ once per timeframe	Order books change every second; a new 1-minute candle closes every 60 s. Fetching OHLCV on every book poll would exhaust rate limits within minutes.
Pre-seeded mock history	Strategies require N bars of history before producing valid signals. Running 100 ticks before the first signal would make mock tests slow and misleading.

4.1 Purpose

4.2 Logic Flowchart



◆

Decision	Reason
Per-handler exception isolation	A bug in the dashboard publisher should not crash the risk manager or stop order flow.
<code>engine.halt()</code> drops ORDER events	The halt is the last-resort safety valve. It does not stop the loop (regime/portfolio still update so the dashboard stays live) — it only prevents new orders from being sent.
Finite producer → clean exit in mock mode	<code>run()</code> monitors <code>all(t.done() for t in producer_tasks)</code> and <code>queue.empty()</code> so mock mode exits automatically after N ticks without a Ctrl-C.

[illegible]

Decision	Reason
Z-score for trend, EWMA vol ratio for volatility	Z-score captures whether the most recent price is statistically displaced from its recent mean (directional drift). EWMA vol ratio detects whether volatility is abnormally high <i>relative to its own recent baseline</i> , not just in absolute terms — so a normally volatile asset doesn't permanently show as HIGH_VOL.
EWMA baseline ($\alpha=0.1$) rather than a fixed threshold	Crypto volatility varies enormously by market phase. A fixed vol threshold would either flag too many normal days as high-vol or miss genuine spikes.
Regime stored in <code>_regime[symbol]</code> dict (not emitted as event for strategy use)	The StrategyManager's <code>on_market</code> runs immediately after the detector's <code>on_market</code> . Reading the dict directly (instead of waiting for a <code>RegimeEvent</code> to be routed through the queue) ensures the strategy always sees the up-to-date regime for the current tick.
HMM upgrade planned (TODO)	GaussianHMM with entropy features would model regime transitions probabilistically, reducing whipsaws at regime boundaries. Current z-score/vol approach serves as a validated baseline.

6. Strategy Layer

Files: `src/strategy/base.py`, `src/strategy/manager.py`,
 `src/strategy/mean_reversion.py`, `src/strategy/momentum.py`,
 `src/utils/indicators.py`

Owner: Cheng, Gilbert

6.1 Purpose

Two concrete strategies handle two different market regimes. A StrategyManager selects the appropriate one per tick, throttles signal cadence, and keeps all strategies warm even when they are not active.

6.2 StrategyManager Logic

```
on_market(MarketEvent)
    ■
    strat.observe(event) ← called on ALL strategies every tick
    (builds price history so each strategy is warm when its
     regime becomes active – no delayed warm-up on regime change)
    ■
    now - last_signal_ts[symbol] < signal_interval (60 s)?
    ■ YES → return None (throttle: one signal per minute)
    ■ NO
    ▼
    regime = detector.current_regime(symbol)
    ■
    Find first strategy where regime ∈ strategy.suitable_regimes
    ■ None found → return None
    ■
    signal = strategy.generate(symbol, price)
    ■
    signal is None or HOLD? → return None
    ■
    Update last_signal_ts[symbol] = now
    Return [SignalEvent]
```

6.3 Momentum Strategy (TRENDING regime)

Indicators: Fast MA (10-bar) + Slow MA (30-bar) + MACD histogram

```

generate(symbol, price)
    fast_ma = moving_average(prices, 10)
    slow_ma = moving_average(prices, 30)
    _, _, hist = macd(prices, 12, 26, 9)

    Any NaN? → return None (insufficient history)

    fast > slow AND hist > 0? → BUY
    fast < slow AND hist < 0? → SELL
    otherwise → HOLD

    magnitude = min(|hist| / (price × 0.001), 1.0)
    (a histogram move equal to 0.1% of price → full conviction)
    final magnitude = 0.4 + 0.6 × computed_mag
    (base floor of 0.4 so any valid signal has non-trivial size)

```

6.4 Mean Reversion Strategy (RANGING / HIGH_VOL regimes)

Indicators: RSI (14-bar) + Bollinger Bands (20-bar, $\pm 2\sigma$)

```

generate(symbol, price)
    rsi = rsi(prices, 14)
    lower, mid, upper = bollinger(prices, 20, 2σ)

    Any NaN? → return None

    price ≤ lower AND rsi ≤ 30? → BUY mag = 0.4 + 0.6 × (30 - rsi) / 30
    price ≥ upper AND rsi ≥ 70? → SELL mag = 0.4 + 0.6 × (rsi - 70) / 30
    otherwise → HOLD

```

6.5 Indicator Implementation Notes

All indicators are implemented in `src/utils/indicators.py` using NumPy directly (no TA-Lib or pandas-ta dependency). They accept any sequence and return the latest scalar value, or NaN when there is insufficient history. This makes them trivially testable and swappable behind the same function signatures.

Indicator	Implementation note
RSI	Wilder's method: mean of N gains / mean of N losses over last window+1 bars

Indicator	Implementation note
Bollinger	Rolling mean $\pm 2 \times$ sample std over last window bars
MACD	Three full EMAs over the entire price series (not a rolling window trick)
Z-score	(last – mean) / sample std over last window bars
Realized vol	Std of log returns over last window bars

6.6 Rationale

Decision	Reason
Two strategies, regime-gated	Momentum and mean reversion are adversarial: applying mean reversion in a trend causes repeated stop-outs; applying momentum in a ranging market causes whipsaws. Regime gating routes each signal type to the market condition it was designed for.
Both require two independent confirmations	Momentum: MA crossover AND positive MACD histogram. Mean reversion: price outside band AND RSI at extreme. A single indicator generates too many false signals; both agreeing provides a much higher-quality trigger.
Conviction magnitude (0.4–1.0)	The Risk Manager uses magnitude to scale position size. The 0.4 floor ensures even a low-conviction signal results in a meaningful (not dust-sized) order.
60-second signal throttle	The market data polls every 10 seconds. Generating an order every 10 seconds on the same symbol would rapidly exhaust the per-symbol position cap and trigger the cooldown. One signal per minute is a natural fit for 1-minute kline strategies.

7. Risk Manager

File: `src/part5_risk/risk_manager.py`

Owner: Gilbert, Grace

7.1 Purpose

The Risk Manager sits between signals and orders. It is the single point where all risk policy is enforced: position sizing, kill-switches, drawdown halt, cooldowns, position caps, funding checks, take-profit/stop-loss exits, and inventory rebalancing.

7.2 Signal → Order Logic (`on_signal`)

```

on_signal(SignalEvent)
    ■
    _update_drawdown() → engine halted? YES → return None
    ■ NO
    control.killed? YES → return None
    ■ NO
    signal == HOLD? YES → return None
    ■ NO
    trading_enabled? (config or dashboard button)
    ■ NO → log, return None
    ■ YES
    price <= 0? → return None
    ■
    Entry cooldown: (now - last_order_ts[symbol]) < cooldown_sec?
    ■ YES → return None
    ■ NO
    side = BUY if signal.BUY else SELL
    ■
    side == BUY:
        max_open_positions cap reached AND new symbol? → return None
        ■
        qty = _size(price) [see sizing modes below]
        ■
    side == SELL:
        held_qty = portfolio.positions[symbol].qty
        held_qty ≤ 0? → return None (spot: no shorting)
        qty = min(qty, held_qty)
        ■
    side == BUY:
        per-symbol notional cap:
        held_notional = pos.qty × price
        room = cap - held_notional
        room ≤ 0? → return None
        qty = min(qty, room / price)
        ■
        funding check:
        cost = qty × price > portfolio.cash?
        → qty = portfolio.cash / price
        qty ≤ 0? → return None
        ■
    qty × price < min_notional_usdt? → return None (dust)
    ■
    Build Order (MARKET or LIMIT at offset_bps inside mid)
    Update last_order_ts[symbol] = now
    Return [OrderEvent]

```

7.3 Sizing Modes

```

mode="notional" → qty = notional_usdt / price (default: 100 USDT)
mode="base" → qty = base_qty (fixed coin amount)
mode="pct_equity" → qty = pct_equity × equity / price (% of portfolio)
mode="allin" → qty = cash × (1 - buffer) / price (all free cash)

All modes → qty = min(qty, max_position_value/price, max_position_qty)

```

[illegible]

Decision	Reason
Single point of risk policy	All gating lives in one class so it is straightforward to audit, test, and tune. No risk logic is scattered across strategy or OMS code.
TP/SL bypasses kill-switch and cooldown	A held position must always be closeable regardless of whether new entries are paused. Blocking exits is a more dangerous failure mode than allowing one extra entry.
Per-symbol position cap (total notional) separate from per-order cap	Per-order caps prevent a single large order but not slow accumulation across many small ones. The per-symbol total cap (<code>max_position_value_per_symbol</code>) is the only mechanism that stops this.
Spot-only enforcement (no shorting)	Spot trading has no borrowed positions. A SELL signal with no long inventory is silently skipped rather than raising an error, since the strategy does not know the current position state.
Inventory balancing as complement to TP/SL	TP/SL protect against loss. Inventory balancing trims an over-weight position that has <i>gained</i> too much, reducing concentration risk without waiting for a reversal.

8. Order Management System

Files: src/part3_execution/oms.py, src/part3_execution/broker.py

Owner: sookoon

8.1 Purpose

The OMS is the boundary between the bot's internal order representation and the actual trading venue. It tracks live LIMIT orders, enforces TTL cancellation, handles the hard-halt cancel-all, and translates broker fills into FillEvent objects.

8.2 OMS Logic Flowchart

```

on_order(OrderEvent)
    ■
    order.type == LIMIT?
    ■ YES → set ttl, add to _open_orders dict
    ■
    fill = broker.place(order, ref_price)
    ■
    fill is None? → mark order REJECTED, return None
    ■
    filled_qty = fill["qty"]
    ■
    filled_qty ≤ 0 (LIMIT resting, not immediately filled)?
    ■ YES → order stays in _open_orders for TTL sweep
    ■     return None (no FillEvent yet)
    ■ NO
    order.filled_qty = filled_qty
    order.status = FILLED or PARTIAL
    FILLED? → remove from _open_orders
    ■
    Return [FillEvent(symbol, side, qty, price, fee)]

```

```

=====

on_market(MarketEvent) ← TTL sweep + halt cancel-all
    ■
    Update _last_price[symbol]
    ■
    engine.halted AND _open_orders not empty?
    ■ YES → cancel_all() → return
    ■ NO
    For each open LIMIT order:
    ■
        order.type == LIMIT AND order.ttl set?
        (now - created_ts) ≥ ttl?
        ■ YES → broker.cancel(order)
            mark CANCELED, remove from _open_orders
            log "limit TTL expired"

```

8.3 Broker Comparison

	MockBroker	CcxtBroker
MARKET fill	Instant, at ref_price	<code>create_order("market", ...)</code> on paper venue
LIMIT fill	Instant, at limit_price	Places on paper book; filled=0 if resting
Cancel	No-op (always True)	<code>cancel_order(exchange_id)</code>
Fee	Configurable taker_fee (0.1%)	Reported by exchange (actual fee in fill response)
Hard guard	N/A	<code>_assert_paper()</code> refuses production endpoint
Credentials	None	BINANCE_TESTNET_API_KEY / BINANCE_DEMO_API_KEY

8.4 Rationale

Decision	Reason
OMS tracks open LIMIT orders; MARKET orders do not rest	MARKET orders fill immediately and need no lifecycle management. LIMIT orders may rest on the book; they need a reference in <code>_open_orders</code> for TTL sweeping and the cancel-all halt.
TTL swept in <code>on_market</code> (not a background timer)	The engine is single-threaded (asyncio). Using <code>on_market</code> to sweep TTLs keeps all state mutations on the event loop, avoiding race conditions.
<code>_assert_paper()</code> in <code>CcxtBroker.__init__</code>	This is a fail-closed safety check: if the sandbox setup fails for any reason and the spot endpoint still points to <code>api.binance.com</code> , the broker raises immediately rather than placing a real order.
<code>filled_qty=0</code> means resting LIMIT (no <code>FillEvent</code>)	A fill event with zero quantity would cause the portfolio to book a zero-size trade. Returning <code>None</code> keeps the order open and lets the TTL sweep decide its fate.

9. Portfolio

File: src/portfolio/portfolio.py

9.1 Purpose

The Portfolio maintains the authoritative record of cash, positions (with average-cost accounting), realized and unrealized PnL, and equity. It is a pure accounting module — it records what happened, it does not decide what to do.

9.2 Logic Flowchart

```
on_market(MarketEvent) ← FIRST MARKET handler
    ■
    _last_price[symbol] = price
    positions[symbol].mark(price) ← update last_price for MTM
    return None
```

```
on_fill(FillEvent)
    ■
    notional = qty × price
    ■
    side == BUY: cash -= notional + fee
    side == SELL: cash += notional - fee
    ■
    position.apply_fill(side, qty, price)
    ■
    If opening / adding in same direction:
        new_avg = (old_avg × |old_qty| + price × qty) / |new_qty|
    ■
    If reducing / flipping:
        realized = (price - avg_price) × closing_qty × sign(qty)
        realized_pnl += realized
        If qty flips through zero → avg_price = price (new entry)
    ■
    Log FILL line
```

```
equity = cash + Σ (position.qty × last_price[symbol])
```

9.3 Rationale

Decision	Reason
Average-cost accounting (not FIFO)	Average-cost is simpler, stateless, and sufficient for the bot's single-lot-per-symbol approach. FIFO would matter for tax purposes on a real account.

Decision	Reason
Mark registered as FIRST MARKET handler	Equity and unrealized PnL must be current before the Risk Manager checks drawdown. Any ordering where portfolio marks <i>after</i> risk checks would compute drawdown on stale prices.
$\text{equity} = \text{cash} + \text{position_value}$ (not realized PnL basis)	This is the true liquidation value of the account, including paper gains and losses in open positions. It is the correct base for drawdown and sizing calculations.

Owner: Cheng, Gilbert

10.2 IPC Architecture


```

render() called by Streamlit runtime
  ■
  _render_controls(st) ← reads control.json; buttons write control.json
  ■
  st.radio("View", ["Overview", "BTC/USDT", "ETH/USDT"])
  ■
  @st.fragment(run_every="2s"):
  ■
    state = load_state(runtime/state.json)
    ■
    state is None?
    → show "start the bot" warning
    ■
    age > 15 s? → show "stale" warning
    ■
    view == "Overview"?
    → _render_overview: metrics row, equity curve line chart,
      positions table, regime badges
    ■
    view == symbol?
    → _render_symbol: price/PnL metrics, Plotly candlestick,
      strategy analytics (BUY/SELL/HOLD counts, last signal JSON)
    ■
    _render_alerts: rolling feed from logging stream

```

10.4 StatePublisher Alert Tap

The StatePublisher installs a custom logging.Handler on the root logger at startup. It filters to:

- WARNING and above from any logger
- INFO from the risk, oms, broker, and control loggers (trade lifecycle events)

This gives the dashboard a live stream of every order, fill, TP/SL trigger, drawdown alert, and kill-switch action — without any direct coupling between those modules and the dashboard.

10.5 Rationale

Decision	Reason
Separate process, JSON IPC	Streamlit has its own event loop and re-execution model that is incompatible with the bot's asyncio loop. Separate processes with a shared file are the simplest, most robust decoupling. Redis or a socket would be more performant but unnecessary for a 2-second refresh interval.
Atomic tmp → rename for state writes	If the dashboard reads the file mid-write, it sees a partial JSON and crashes. The atomic rename guarantees the reader always sees a complete, valid snapshot.
st.fragment(run_every="2s")	Only the data display reruns every 2 s; the control buttons and view selector live outside the fragment so a button click causes a full rerun rather than being overwritten by the next auto-refresh.

Decision	Reason
Alert tap via logging.Handler	Avoids adding any dashboard-awareness to the risk, OMS, or broker modules. Those modules log as they always would; the dashboard passively captures it.

11. Backtester

Files: src/part8_backtest/engine.py, src/part8_backtest/data.py,
src/part8_backtest/metrics.py, src/part8_backtest/app.py

11.1 Purpose

The backtester replays historical OHLCV data through the **identical** strategy, regime, portfolio, and risk modules that run live. No special backtesting versions of these modules exist — the same code is exercised by both live trading and backtesting.

11.2 Logic Flowchart

```
run_backtest(symbol, candles, cfg, ...)
    ■
    Build: Portfolio, Engine, RegimeDetector,
           StrategyManager, RiskManager, MockBroker
    ■
    For each candle c[i]:
        ■
        me = MarketEvent(price=c.close, candles=c[0:i+1 bounded to window])
        ■
        portfolio.on_market(me)           ← mark positions
        ■
        for ev in risk.on_market(me):   ← TP/SL exits
            _execute(ev.order, c.close)
        ■
        detector.on_market(me)           ← update regime
        ■
        for sig in manager.on_market(me): ← generate signals
            for oe in risk.on_signal(sig): ← size orders
                _execute(oe.order, c.close)
        ■
        Append (ts, equity) to equity_curve
        Append (ts, regime) to regime_series
        ■
        engine.halted? → result.halted = True
        ■
        compute_metrics(result) →
            total return, buy-and-hold return,
            max drawdown, annualised Sharpe,
            win rate, profit factor, trade count, fees
        ■
    Return BacktestResult
```

11.3 Rationale

Decision	Reason
Same modules, no backtesting wrappers	The most common failure mode in backtesting is "backtest code and live code diverge." Using the real modules by construction makes any live strategy bug visible in the backtest and vice versa.
Fills at bar close	Bar-level backtests cannot know the intrabar path. Filling at close is conservative (avoids look-ahead on high/low) and consistent across all order types.
Bounded trailing window (<code>kline_limit</code>)	The live <code>CcxtConnector</code> only ever hands strategies the last 100 bars. The backtest enforces the same window so indicator values match what the live bot would compute — preventing backtest overfitting from seeing more history than the live strategy would.
Standalone CLI + web UI	The backtester is fully independent of <code>main.py</code> and the live dashboard. This means it can be run offline without credentials and doesn't interfere with a running bot.

12. Key Design Decisions

12.1 Event-Driven Architecture

Decision: All inter-module communication happens through typed events in a single queue. Modules never call each other.

Rationale: Decoupling allows any module to be replaced, tested in isolation, or disabled without touching other modules. New modules (a second exchange, a third strategy) are added by registering a handler — no existing code changes. The queue also naturally serialises concurrent data from multiple symbols, avoiding shared-state race conditions.

12.2 Testnet-Only Constraint

Decision: Two independent safety mechanisms prevent real orders: (1) the `CcxtBroker._assert_paper()` hard guard that refuses to operate against `api.binance.com`, and (2) a data connector built without API credentials that physically cannot authenticate to place orders.

Rationale: A single configuration flag (`sandbox=True`) is a single point of failure. Belt-and-suspenders enforcement makes accidental real trading impossible even if one mechanism is misconfigured.

12.3 Configuration as Single Source of Truth

Decision: All tunable knobs (sizing mode, TP/SL percentages, cooldown, position caps, order type) live in `config/config.yaml`'s `order: block`. No code changes are needed to tune strategy behaviour.

Rationale: Trading strategy parameters need rapid iteration. Keeping them in config prevents accidental code changes during tuning sessions and makes the parameter set visible and reviewable in one place.

12.4 Handler Registration Order as a Contract

Decision: The order in which handlers are registered in `main.py` is documented and treated as load-bearing. Comments in the code explicitly flag it.

Rationale: Several correctness invariants depend on ordering: equity must be marked before drawdown is checked; regime must be classified before strategy generates a signal; publisher must snapshot after all state changes. Reordering silently breaks these invariants — making the ordering explicit prevents it.

13. Current Status and Roadmap

Completed and Verified (Mock Mode)

Component	Status
Data layer (Mock + Ccxt connectors)	Complete
Async event engine	Complete
Z-score/vol regime detector	Complete
Momentum strategy (MACD + MA)	Complete
Mean reversion strategy (RSI + Bollinger)	Complete
Portfolio (average-cost, MTM, equity)	Complete
Risk sizing (4 modes), drawdown halt, kill-switch	Complete
OMS (market fill, limit TTL, cancel-all)	Complete
Dashboard IPC + controls + live charts	Complete
Backtester (CLI + web, all metrics)	Complete
Testnet / Demo Trading broker wiring	Code complete; live verification pending

Pending (Roadmap)

Step	Work Remaining	Owner
Step 1	Verify live data feed (python scripts/check_data.py)	All
Step 2	30-min observe-only run: log regimes + signals, no orders	Cheng, ShiYi

Step	Work Remaining	Owner
Step 3	First live paper order; confirm fill + balance update	sookoon
Step 4	Partial fills; inventory balancing on live orders	sookoon
Step 5	Historical VaR (10-min window); daily loss halt; trailing stop	Gilbert, Grace
Step 6	Replace heuristic with GaussianHMM + entropy feature	ShiYi
Step 7	Dashboard live wiring (verify with running bot)	Cheng, Gilbert
Step 8	Full backtest validation against historical data	All
Step 9	Class diagram + 5-minute pitch	All

End of report. Generated 2026-06-14.